



Admitix SQL schema code

```
-- =====  
=====  
-- ADMITIX – Admission Workflow Management SaaS Platform  
-- Production Database Schema (PostgreSQL 13+) – FIXED VERSION  
-- =====  
=====  
-- This is your original 23-table schema with ONLY mandatory  
-- corrections applied. No new tables were introduced. Every  
change  
-- is marked with a "-- FIX:" comment explaining why.  
-- =====  
=====
```

```
BEGIN;
```

```
-- =====  
=====  
-- 0. EXTENSIONS  
-- =====  
=====  
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

```
-- =====  
=====
```

```
-- 0.1 SHARED TRIGGER FUNCTION: auto-update updated_at
-- =====
=====
CREATE OR REPLACE FUNCTION trigger_set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
NEW.updated_at = now();
RETURN NEW;
END;
```

```
--LANGUAGE plpgsql;

-- =====
=====
-- 1. ROLES
-- =====
=====
CREATE TABLE roles (
    role_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    role_name      VARCHAR(50) NOT NULL UNIQUE,
    description     TEXT,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);
COMMENT ON TABLE roles IS 'RBAC roles: super_admin, instituti
on_admin, admission_officer, department_reviewer, finance_off
icer, registrar, faculty, student, guardian.';

-- =====
=====
-- 2. INSTITUTIONS (tenants)
-- =====
=====
CREATE TABLE institutions (
    institution_id  UUID PRIMARY KEY DEFAULT gen_random_uui
d(),
```

```

institution_name  VARCHAR(150) NOT NULL,
institution_code  VARCHAR(20)  NOT NULL,
email            VARCHAR(150) NOT NULL,
phone            VARCHAR(20),
address          TEXT,
city             VARCHAR(100),
state            VARCHAR(100),
country          VARCHAR(100) NOT NULL DEFAULT 'India',
logo_url         TEXT,
status           BOOLEAN NOT NULL DEFAULT TRUE,
created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
CONSTRAINT uq_institutions_code  UNIQUE (institution_cod
e),
CONSTRAINT uq_institutions_email UNIQUE (email)
);

```

```

CREATE TRIGGER set_updated_at_institutions
BEFORE UPDATE ON institutions
FOR EACH ROW EXECUTE FUNCTION trigger_set_updated_at();

```

```

CREATE INDEX idx_institutions_status ON institutions (statu
s);

```

```

-- =====
=====
-- 3. FACULTIES
-- FIX: institution_id FK changed CASCADE -> RESTRICT.
-- Reason: deleting an institution should never be able to si
lently
-- wipe every faculty/department/course/student/payment benea
th it.
-- Institutions are deactivated via `status = false`, not del
eted.
-- If a hard delete is ever truly needed, the app must explic
itly

```

```

-- delete children first (or you deliberately drop this RESTR
ICT).
-- =====
=====
CREATE TABLE faculties (
    faculty_id      UUID PRIMARY KEY DEFAULT gen_random_uuid
    ( ),
    institution_id  UUID NOT NULL REFERENCES institutions (in
stitution_id) ON DELETE RESTRICT,
    faculty_name    VARCHAR(150) NOT NULL,
    description     TEXT,
    status          BOOLEAN NOT NULL DEFAULT TRUE,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT uq_faculties_institution_name UNIQUE (institut
ion_id, faculty_name)
);

CREATE INDEX idx_faculties_institution_id ON faculties (insti
tution_id);

-- =====
=====
-- 4. DEPARTMENTS
-- FIX: hod_name (free text) replaced with hod_staff_id, a re
al FK
-- to staff. Reason: HOD must be a verifiable staff member, n
ot an
-- unvalidated string. staff table doesn't exist yet at this
point
-- in script order, so the FK constraint is added later via A
LTER
-- TABLE (see section 8.1) to avoid a circular dependency.
-- =====
=====
CREATE TABLE departments (
    department_id   UUID PRIMARY KEY DEFAULT gen_random_uuid

```

```
(
    faculty_id        UUID NOT NULL REFERENCES faculties (faculty_id) ON DELETE CASCADE,
    institution_id     UUID NOT NULL REFERENCES institutions (institution_id) ON DELETE RESTRICT,
    department_name    VARCHAR(150) NOT NULL,
    hod_staff_id       UUID, -- FK added in section 8.1 after `staff` exists
    description        TEXT,
    status             BOOLEAN NOT NULL DEFAULT TRUE,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT uq_departments_faculty_name UNIQUE (faculty_id, department_name)
);
```

```
CREATE INDEX idx_departments_faculty_id ON departments (faculty_id);
```

```
CREATE INDEX idx_departments_institution_id ON departments (institution_id);
```

```
CREATE INDEX idx_departments_hod_staff_id ON departments (hod_staff_id);
```

```
-- =====
=====
```

```
-- 5. COURSES
```

```
-- =====
=====
```

```
CREATE TABLE courses (
    course_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    department_id      UUID NOT NULL REFERENCES departments (department_id) ON DELETE CASCADE,
    institution_id     UUID NOT NULL REFERENCES institutions (institution_id) ON DELETE RESTRICT,
    course_name        VARCHAR(150) NOT NULL,
    course_code        VARCHAR(30) NOT NULL,
```

```

        duration_years SMALLINT NOT NULL CHECK (duration_years >
0),
        eligibility TEXT,
        total_seats INTEGER NOT NULL DEFAULT 0 CHECK (total_s
eats >= 0),
        status BOOLEAN NOT NULL DEFAULT TRUE,
        created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
        CONSTRAINT uq_courses_institution_code UNIQUE (institutio
n_id, course_code)
    );
COMMENT ON COLUMN courses.total_seats IS 'Kept in sync automa
tically with SUM(seat_matrix.total_seats) for this course via
trigger – see section 8.3. Do not update manually.';

CREATE INDEX idx_courses_department_id ON courses (departme
nt_id);
CREATE INDEX idx_courses_institution_id ON courses (institut
ion_id);
CREATE INDEX idx_courses_status ON courses (status);

-- =====
--
-- 6. ADMISSION CYCLES
-- =====
CREATE TABLE admission_cycles (
    cycle_id UUID PRIMARY KEY DEFAULT gen_random_uu
id(),
    institution_id UUID NOT NULL REFERENCES institutions
(institution_id) ON DELETE RESTRICT,
    academic_year VARCHAR(20) NOT NULL,
    application_start DATE NOT NULL,
    application_end DATE NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'upcomin
g'
    CHECK (status IN ('upcoming', 'open',

```

```

'closed', 'archived'))),
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT uq_admission_cycles_institution_year UNIQUE (i
nstitution_id, academic_year),
    CONSTRAINT ck_admission_cycles_dates CHECK (application_e
nd > application_start)
);

CREATE INDEX idx_admission_cycles_institution_id ON admission
_cycles (institution_id);
CREATE INDEX idx_admission_cycles_status          ON admission
_cycles (status);

-- =====
--
-- 7. USERS
-- FIX: global UNIQUE(email) replaced with two partial unique
indexes:
--   (a) unique per institution for tenant users
--   (b) unique globally only among institution_id IS NULL (s
uper admins)
-- Reason: a student applying to two different colleges on th
is
-- platform needs a separate login per institution, but shoul
d be
-- able to reuse the same email address across institutions.
-- =====
--
CREATE TABLE users (
    user_id          UUID PRIMARY KEY DEFAULT gen_random_uuid
(),
    institution_id   UUID REFERENCES institutions (institution
_id) ON DELETE RESTRICT,
    role_id          UUID NOT NULL REFERENCES roles (role_id)
ON DELETE RESTRICT,
    first_name       VARCHAR(100) NOT NULL,

```

```

    last_name      VARCHAR(100),
    email          VARCHAR(150) NOT NULL,
    phone          VARCHAR(20),
    password_hash  TEXT NOT NULL,
    profile_photo  TEXT,
    is_active      BOOLEAN NOT NULL DEFAULT TRUE,
    last_login     TIMESTAMPTZ,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);
COMMENT ON COLUMN users.institution_id IS 'NULL for platform-
level Super Admin users; required for all tenant users.';

-- FIX: replaces `uq_users_email UNIQUE (email)`
CREATE UNIQUE INDEX uq_users_institution_email
    ON users (institution_id, email)
    WHERE institution_id IS NOT NULL;

CREATE UNIQUE INDEX uq_users_super_admin_email
    ON users (email)
    WHERE institution_id IS NULL;

CREATE TRIGGER set_updated_at_users
BEFORE UPDATE ON users
FOR EACH ROW EXECUTE FUNCTION trigger_set_updated_at();

CREATE INDEX idx_users_institution_id ON users (institution_i
d);
CREATE INDEX idx_users_role_id      ON users (role_id);
CREATE INDEX idx_users_is_active   ON users (is_active);

-- =====
=====
-- 8. STAFF
-- =====
=====

```



```

CREATE TABLE staff (
    staff_id          UUID PRIMARY KEY DEFAULT gen_random_uuid
    ( ),
    user_id           UUID NOT NULL REFERENCES users (user_id)
ON DELETE CASCADE,
    institution_id    UUID NOT NULL REFERENCES institutions (in
stitution_id) ON DELETE RESTRICT,
    department_id     UUID REFERENCES departments (department_i
d) ON DELETE SET NULL,
    employee_id       VARCHAR(50) NOT NULL,
    designation       VARCHAR(100),
    joining_date      DATE,
    status            BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT uq_staff_user_id          UNIQUE (user_id),
    CONSTRAINT uq_staff_institution_empid UNIQUE (instituti
on_id, employee_id)
);

```

```

CREATE INDEX idx_staff_institution_id ON staff (institution_i
d);
CREATE INDEX idx_staff_department_id  ON staff (department_i
d);

```

```

-- =====
=====

```

```

-- 8.1 FIX: add the deferred FK from departments.hod_staff_id
-> staff
-- (staff didn't exist yet when departments was created abov
e)

```

```

-- =====
=====

```

```

ALTER TABLE departments
    ADD CONSTRAINT fk_departments_hod_staff
    FOREIGN KEY (hod_staff_id) REFERENCES staff (staff_id) ON
DELETE SET NULL;

```

```

-- =====
=====
-- 8.2 FIX: trigger to guarantee staff.institution_id always
matches
-- the institution_id of its own parent user row.
-- Reason: institution_id is duplicated on staff/students for
query
-- speed, but nothing previously stopped it drifting out of s
ync
-- with the user it belongs to (a tenant-isolation bug waitin
g to
-- happen). This trigger rejects any insert/update that disag
rees.
-- =====
=====
CREATE OR REPLACE FUNCTION check_staff_institution_matches_us
er()
RETURNS TRIGGER AS

```

```

DECLARE
user_institution UUID;
BEGIN
SELECT institution_id INTO user_institution FROM users WHERE
user_id = NEW.user_id;
IF user_institution IS DISTINCT FROM NEW.institution_id THEN
RAISE EXCEPTION 'staff.institution_id (%) does not match user
s.institution_id (%) for user_id %',
NEW.institution_id, user_institution, NEW.user_id;
END IF;
RETURN NEW;
END;

```

```

LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_check_staff_institution
BEFORE INSERT OR UPDATE ON staff
FOR EACH ROW EXECUTE FUNCTION check_staff_institution_matches
_user();

-- =====
=====
-- 9. STUDENTS
-- =====
=====
CREATE TABLE students (
    student_id      UUID PRIMARY KEY DEFAULT gen_random_uuid
    (),
    user_id          UUID NOT NULL REFERENCES users (user_id)
ON DELETE CASCADE,
    institution_id   UUID NOT NULL REFERENCES institutions (in
stitution_id) ON DELETE RESTRICT,
    aadhaar_no       VARCHAR(20),
    gender           VARCHAR(20) CHECK (gender IN ('male', 'fe
male', 'other', 'prefer_not_to_say')),
    dob              DATE,
    blood_group       VARCHAR(5),
    category          VARCHAR(30),
    nationality        VARCHAR(50) NOT NULL DEFAULT 'Indian',
    address           TEXT,
    city              VARCHAR(100),
    state             VARCHAR(100),
    pincode           VARCHAR(10),
    parent_name       VARCHAR(150),
    parent_phone      VARCHAR(20),
    guardian_email    VARCHAR(150),
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT uq_students_user_id UNIQUE (user_id)
);

CREATE UNIQUE INDEX uq_students_institution_aadhaar

```

```
ON students (institution_id, aadhaar_no)
WHERE aadhaar_no IS NOT NULL;
```

```
CREATE INDEX idx_students_institution_id ON students (institution_id);
```

```
-- FIX: same tenant-isolation guarantee as staff, applied to students.
```

```
CREATE OR REPLACE FUNCTION check_student_institution_matches_user()
RETURNS TRIGGER AS
```

```
DECLARE
user_institution UUID;
BEGIN
SELECT institution_id INTO user_institution FROM users WHERE
user_id = NEW.user_id;
IF user_institution IS DISTINCT FROM NEW.institution_id THEN
RAISE EXCEPTION 'students.institution_id (%) does not match users.institution_id (%) for user_id %',
NEW.institution_id, user_institution, NEW.user_id;
END IF;
RETURN NEW;
END;
```

```
LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_check_student_institution
BEFORE INSERT OR UPDATE ON students
FOR EACH ROW EXECUTE FUNCTION check_student_institution_matches_user();
```

```
-- =====
=====
```

```

-- 10. EDUCATION DETAILS
-- =====
=====
CREATE TABLE education_details (
    education_id      UUID PRIMARY KEY DEFAULT gen_random_uui
d(),
    student_id        UUID NOT NULL REFERENCES students (stud
ent_id) ON DELETE CASCADE,
    qualification      VARCHAR(100) NOT NULL,
    board_university  VARCHAR(150),
    institution_name   VARCHAR(150),
    passing_year       SMALLINT CHECK (passing_year BETWEEN 19
50 AND 2100),
    seat_number        VARCHAR(50),
    percentage         DECIMAL(5,2) CHECK (percentage BETWEEN
0 AND 100),
    cgpa               DECIMAL(4,2) CHECK (cgpa BETWEEN 0 AND
10),
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_education_details_student_id ON education_de
tails (student_id);

-- =====
=====
-- 11. ENTRANCE EXAM SCORES
-- =====
=====
CREATE TABLE entrance_exam_scores (
    score_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    student_id        UUID NOT NULL REFERENCES students (student_i
d) ON DELETE CASCADE,
    exam_name         VARCHAR(100) NOT NULL,
    roll_number        VARCHAR(50),
    score             DECIMAL(8,2),

```

```

        percentile    DECIMAL(5,2) CHECK (percentile BETWEEN 0 AND
100),
        rank          INTEGER CHECK (rank > 0),
        exam_year     SMALLINT CHECK (exam_year BETWEEN 1950 AND 2
100),
        created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

```

CREATE INDEX idx_entrance_exam_scores_student_id ON entrance_
exam_scores (student_id);

```

```

-- =====
=====
-- 12. APPLICATIONS
-- =====
=====

```

```

CREATE TABLE applications (
    application_id    UUID PRIMARY KEY DEFAULT gen_random_u
uid(),
    student_id       UUID NOT NULL REFERENCES students (st
udent_id) ON DELETE CASCADE,
    cycle_id         UUID NOT NULL REFERENCES admission_cy
cles (cycle_id) ON DELETE RESTRICT,
    application_number VARCHAR(30) NOT NULL,
    submission_date   TIMESTAMPTZ NOT NULL DEFAULT now(),
    current_status    VARCHAR(30) NOT NULL DEFAULT 'draft'
                    CHECK (current_status IN (
                        'draft', 'submitted', 'under_rev
iew', 'documents_pending',
                        'approved', 'rejected', 'admitte
d', 'cancelled'
                    )),
    reviewed_by      UUID REFERENCES users (user_id) ON DELETE SE
T NULL,
    remarks          TEXT,
    created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),

```

```

        CONSTRAINT uq_applications_number        UNIQUE (applica
tion_number),
        CONSTRAINT uq_applications_student_cycle UNIQUE (student
_id, cycle_id)
    );

CREATE INDEX idx_applications_student_id        ON applications
(student_id);
CREATE INDEX idx_applications_cycle_id          ON applications
(cycle_id);
CREATE INDEX idx_applications_reviewed_by       ON applications
(reviewed_by);
CREATE INDEX idx_applications_current_status    ON applications
(current_status);

-- =====
=====
-- 13. APPLICATION PREFERENCES
-- =====
=====
CREATE TABLE application_preferences (
    preference_id    UUID PRIMARY KEY DEFAULT gen_random_uuid
    (),
    application_id   UUID NOT NULL REFERENCES applications (ap
plication_id) ON DELETE CASCADE,
    course_id        UUID NOT NULL REFERENCES courses (course
_id) ON DELETE RESTRICT,
    preference_no     SMALLINT NOT NULL CHECK (preference_no >
0),
    status            VARCHAR(20) NOT NULL DEFAULT 'pending'
                    CHECK (status IN ('pending', 'allotted',
'rejected', 'withdrawn')),
    CONSTRAINT uq_app_preferences_app_order     UNIQUE (applicat
ion_id, preference_no),
    CONSTRAINT uq_app_preferences_app_course    UNIQUE (applicat
ion_id, course_id)

```

```

);

CREATE INDEX idx_app_preferences_application_id ON applicatio
n_preferences (application_id);
CREATE INDEX idx_app_preferences_course_id      ON applicatio
n_preferences (course_id);

-- =====
=====
-- 13.1 FIX: only one preference per application can ever be
'allotted'.
-- Reason: nothing previously stopped two course preferences
on the
-- same application both being marked allotted, which would c
orrupt
-- seat counts and admission records.
-- =====
=====
CREATE UNIQUE INDEX uq_app_preferences_one_allotted
    ON application_preferences (application_id)
    WHERE status = 'allotted';

-- =====
=====
-- 13.2 FIX: keep seat_matrix.filled_seats in sync automatica
lly when
-- a preference becomes/un-becomes 'allotted'. Reason: previo
usly
-- filled_seats had to be updated by hand elsewhere in the ap
p, with
-- nothing to stop it drifting from the actual allotment dat
a.
-- =====
=====
CREATE OR REPLACE FUNCTION sync_seat_matrix_on_preference_cha

```



```
nge()  
RETURNS TRIGGER AS
```

```
DECLARE  
v_category VARCHAR(30);  
BEGIN  
-- Newly allotted: increment filled_seats for that course+category  
IF (TG_OP = 'INSERT' AND NEW.status = 'allotted')  
OR (TG_OP = 'UPDATE' AND NEW.status = 'allotted' AND OLD.status IS DISTINCT FROM 'allotted') THEN  
SELECT category INTO v_category FROM students s  
JOIN applications a ON a.student_id = s.student_id  
WHERE a.application_id = NEW.application_id;  
UPDATE seat_matrix  
SET filled_seats = filled_seats + 1  
WHERE course_id = NEW.course_id AND category = v_category;  
END IF;
```

```
-- Un-allotted (rejected/withdrawn after being allotted): decrement  
IF (TG_OP = 'UPDATE' AND OLD.status = 'allotted' AND NEW.status IS DISTINCT FROM 'allotted') THEN  
SELECT category INTO v_category FROM students s  
JOIN applications a ON a.student_id = s.student_id  
WHERE a.application_id = OLD.application_id;  
UPDATE seat_matrix  
SET filled_seats = filled_seats - 1  
WHERE course_id = OLD.course_id AND category = v_category AND filled_seats > 0;  
END IF;  
  
RETURN NEW;
```

```
END;
```

```
LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_sync_seat_matrix
AFTER INSERT OR UPDATE ON application_preferences
FOR EACH ROW EXECUTE FUNCTION sync_seat_matrix_on_preference_
change();
```

```
-- =====
=====
```

```
-- 14. DOCUMENT TYPES
```

```
-- =====
=====
```

```
CREATE TABLE document_types (
    document_type_id UUID PRIMARY KEY DEFAULT gen_random_uui
d(),
    document_name     VARCHAR(100) NOT NULL UNIQUE,
    mandatory         BOOLEAN NOT NULL DEFAULT TRUE,
    description        TEXT
);
```

```
-- =====
=====
```

```
-- 15. DOCUMENTS
```

```
-- =====
=====
```

```
CREATE TABLE documents (
    document_id        UUID PRIMARY KEY DEFAULT gen_random
_uuid(),
    application_id      UUID NOT NULL REFERENCES applicati
ons (application_id) ON DELETE CASCADE,
    document_type_id    UUID NOT NULL REFERENCES document_
types (document_type_id) ON DELETE RESTRICT,
```

```

        file_name          VARCHAR(255) NOT NULL,
        file_url           TEXT NOT NULL,
        verification_status VARCHAR(20) NOT NULL DEFAULT 'pending'
                                CHECK (verification_status IN (
                                    'pending', 'verified', 'rejected', 'reupload_requested'
                                )),
        verified_by        UUID REFERENCES users (user_id) ON DELETE SET NULL,
        uploaded_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
        remarks            TEXT
    );

CREATE INDEX idx_documents_application_id ON documents
(application_id);
CREATE INDEX idx_documents_document_type_id ON documents
(document_type_id);
CREATE INDEX idx_documents_verified_by ON documents
(verified_by);
CREATE INDEX idx_documents_verification_status ON documents
(verification_status);

-- =====
--
-- 16. AI VERIFICATIONS
-- =====

CREATE TABLE ai_verifications (
    verification_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id        UUID NOT NULL REFERENCES documents (document_id) ON DELETE CASCADE,
    ocr_text           TEXT,
    confidence_score    DECIMAL(5,2) CHECK (confidence_score BETWEEN 0 AND 100),

```

```

        blur_score          DECIMAL(5,2) CHECK (blur_score BETWEEN
0 AND 100),
        missing_fields      TEXT,
        name_match          BOOLEAN,
        status              VARCHAR(20) NOT NULL DEFAULT 'pending'
                           CHECK (status IN ('pending', 'passed',
'failed', 'manual_review')),
        verified_at         TIMESTAMPTZ,
        CONSTRAINT uq_ai_verifications_document_id UNIQUE (docume
nt_id)
    );

```

```

CREATE INDEX idx_ai_verifications_document_id ON ai_verificat
ions (document_id);

```

```

-- =====
=====

```

```

-- 16.1 FIX: auto-propagate AI verification result onto the p
arent
-- document's verification_status, instead of leaving the two
-- columns to drift out of sync (previously nothing linked th
em).
-- A human verifier (admission officer) can still manually ov
erride
-- afterward – this only sets the initial/AI-driven state.

```

```

-- =====
=====

```

```

CREATE OR REPLACE FUNCTION sync_document_status_from_ai()
RETURNS TRIGGER AS

```

```

BEGIN
IF NEW.status = 'failed' THEN
UPDATE documents SET verification_status = 'reupload_requeste
d'
WHERE document_id = NEW.document_id AND verification_status =

```

```

'pending';
ELSIF NEW.status = 'passed' THEN
UPDATE documents SET verification_status = 'verified'
WHERE document_id = NEW.document_id AND verification_status =
'pending';
END IF;
RETURN NEW;
END;

```

```

LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_sync_document_status
AFTER INSERT OR UPDATE ON ai_verifications
FOR EACH ROW EXECUTE FUNCTION sync_document_status_from_ai();

```

```

-- =====
=====

```

```

-- 17. APPLICATION STATUS HISTORY
-- FIX: added institution_id so audit/history queries can filter by
-- tenant directly, instead of requiring a multi-hop join through
-- applications -> students every time. Populated automatically by
-- the trigger below, not by the application layer.
-- =====
=====

```

```

CREATE TABLE application_status_history (
    history_id        UUID PRIMARY KEY DEFAULT gen_random_uuid
    ( ),
    application_id    UUID NOT NULL REFERENCES applications (application_id) ON DELETE CASCADE,
    institution_id    UUID NOT NULL REFERENCES institutions (institution_id) ON DELETE RESTRICT,
    old_status        VARCHAR(30),

```

```

        new_status          VARCHAR(30) NOT NULL,
        changed_by          UUID REFERENCES users (user_id) ON DELETE
SET NULL,
        remarks             TEXT,
        changed_at          TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

```

CREATE INDEX idx_app_status_history_application_id ON applica
tion_status_history (application_id);
CREATE INDEX idx_app_status_history_changed_by      ON applica
tion_status_history (changed_by);
CREATE INDEX idx_app_status_history_institution_id ON applica
tion_status_history (institution_id);

```

```

-- =====
=====
-- 17.1 FIX: auto-fill institution_id on insert instead of tr
usting
-- the app layer to pass it correctly every time.
-- =====
=====
CREATE OR REPLACE FUNCTION set_status_history_institution()
RETURNS TRIGGER AS

```

```

BEGIN
IF NEW.institution_id IS NULL THEN
SELECT s.institution_id INTO NEW.institution_id
FROM applications a JOIN students s ON s.student_id = a.stude
nt_id
WHERE a.application_id = NEW.application_id;
END IF;
RETURN NEW;
END;

```

```

LANGUAGE plpgsql;

CREATE TRIGGER trg_set_status_history_institution
BEFORE INSERT ON application_status_history
FOR EACH ROW EXECUTE FUNCTION set_status_history_institution
();

-- =====
=====
-- 18. FEE STRUCTURE
-- =====
=====
CREATE TABLE fee_structure (
    fee_id          UUID PRIMARY KEY DEFAULT gen_random_uuid
    (),
    course_id       UUID NOT NULL REFERENCES courses (course
_id) ON DELETE CASCADE,
    category        VARCHAR(30) NOT NULL,
    tuition_fee     DECIMAL(12,2) NOT NULL DEFAULT 0 CHECK
(tuition_fee >= 0),
    admission_fee   DECIMAL(12,2) NOT NULL DEFAULT 0 CHECK
(admission_fee >= 0),
    other_fee       DECIMAL(12,2) NOT NULL DEFAULT 0 CHECK
(other_fee >= 0),
    total_fee       DECIMAL(12,2) GENERATED ALWAYS AS (tuiti
on_fee + admission_fee + other_fee) STORED,
    effective_from  DATE NOT NULL,
    CONSTRAINT uq_fee_structure_course_cat_date UNIQUE (cours
e_id, category, effective_from)
);

CREATE INDEX idx_fee_structure_course_id ON fee_structure (co
urse_id);

-- =====

```

```

=====
-- 19. PAYMENTS
-- =====
=====
CREATE TABLE payments (
    payment_id        UUID PRIMARY KEY DEFAULT gen_random_uuid
    (),
    application_id    UUID NOT NULL REFERENCES applications (ap
plication_id) ON DELETE CASCADE,
    fee_id            UUID NOT NULL REFERENCES fee_structure (f
ee_id) ON DELETE RESTRICT,
    amount_paid       DECIMAL(12,2) NOT NULL CHECK (amount_paid
> 0),
    payment_mode      VARCHAR(30) CHECK (payment_mode IN ('onli
ne', 'cash', 'cheque', 'dd', 'card', 'upi')),
    transaction_id    VARCHAR(100),
    payment_status    VARCHAR(20) NOT NULL DEFAULT 'pending'
                    CHECK (payment_status IN ('pending', 'su
ccess', 'failed', 'refunded')),
    payment_date      TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT uq_payments_transaction_id UNIQUE (transaction
_id)
);

CREATE INDEX idx_payments_application_id ON payments (applica
tion_id);
CREATE INDEX idx_payments_fee_id        ON payments (fee_i
d);
CREATE INDEX idx_payments_status        ON payments (payment
_status);

-- =====
=====
-- 20. SEAT MATRIX
-- =====
=====

```



```

CREATE TABLE seat_matrix (
    seat_id          UUID PRIMARY KEY DEFAULT gen_random_uui
d(),
    course_id        UUID NOT NULL REFERENCES courses (cours
e_id) ON DELETE CASCADE,
    category          VARCHAR(30) NOT NULL,
    total_seats       INTEGER NOT NULL CHECK (total_seats >=
0),
    filled_seats      INTEGER NOT NULL DEFAULT 0 CHECK (fille
d_seats >= 0),
    available_seats   INTEGER GENERATED ALWAYS AS (total_sea
ts - filled_seats) STORED,
    CONSTRAINT uq_seat_matrix_course_category UNIQUE (course_
id, category),
    CONSTRAINT ck_seat_matrix_filled_within_total CHECK (fill
ed_seats <= total_seats)
);

```

```

CREATE INDEX idx_seat_matrix_course_id ON seat_matrix (course
_id);

```

```

-- =====
=====

```

```

-- 20.1 FIX: keep courses.total_seats automatically equal to
-- SUM(seat_matrix.total_seats) for that course, instead of t
he two
-- numbers being entered independently and silently drifting
apart.

```

```

-- =====
=====

```

```

CREATE OR REPLACE FUNCTION sync_course_total_seats()
RETURNS TRIGGER AS

```

```

DECLARE
v_course_id UUID;

```

```

BEGIN
v_course_id := COALESCE(NEW.course_id, OLD.course_id);
UPDATE courses
SET total_seats = COALESCE((SELECT SUM(total_seats) FROM seat
_matrix WHERE course_id = v_course_id), 0)
WHERE course_id = v_course_id;
RETURN NULL;
END;

```

```

LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_sync_course_total_seats
AFTER INSERT OR UPDATE OR DELETE ON seat_matrix
FOR EACH ROW EXECUTE FUNCTION sync_course_total_seats();

```

```

-- =====
=====

```

```

-- 21. NOTIFICATIONS

```

```

-- =====
=====

```

```

CREATE TABLE notifications (
    notification_id    UUID PRIMARY KEY DEFAULT gen_random_uu
id(),
    user_id            UUID NOT NULL REFERENCES users (user_i
d) ON DELETE CASCADE,
    title              VARCHAR(200) NOT NULL,
    message            TEXT NOT NULL,
    notification_type   VARCHAR(20) NOT NULL DEFAULT 'in_app'
CHECK (notification_type IN ('email',
'sms', 'in_app')),
    is_read            BOOLEAN NOT NULL DEFAULT FALSE,
    sent_at            TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

```

CREATE INDEX idx_notifications_user_id ON notifications (use

```

```

r_id);
CREATE INDEX idx_notifications_is_read ON notifications (use
r_id, is_read);

-- =====
=====
-- 22. AUDIT LOGS
-- FIX: added institution_id, same reasoning as application_s
tatus_history
-- – direct tenant filtering without joining through users ev
ery time.
-- Nullable because Super Admin actions have no institution.
-- =====
=====
CREATE TABLE audit_logs (
    log_id          UUID PRIMARY KEY DEFAULT gen_random_uuid
()),
    user_id         UUID REFERENCES users (user_id) ON DELETE
SET NULL,
    institution_id  UUID REFERENCES institutions (institution
_id) ON DELETE SET NULL,
    action          VARCHAR(50) NOT NULL,
    table_name      VARCHAR(100) NOT NULL,
    record_id       UUID,
    ip_address      VARCHAR(45),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_audit_logs_user_id          ON audit_logs (use
r_id);
CREATE INDEX idx_audit_logs_table_name      ON audit_logs (tab
le_name);
CREATE INDEX idx_audit_logs_created_at      ON audit_logs (cre
ated_at);
CREATE INDEX idx_audit_logs_institution_id  ON audit_logs (ins
titution_id);

```

```
-- =====
=====
-- 22.1 FIX: auto-fill audit_logs.institution_id from the act
ing user,
-- same pattern as application_status_history.
-- =====
=====
CREATE OR REPLACE FUNCTION set_audit_log_institution()
RETURNS TRIGGER AS
```

```
BEGIN
IF NEW.institution_id IS NULL AND NEW.user_id IS NOT NULL THE
N
SELECT institution_id INTO NEW.institution_id FROM users WHER
E user_id = NEW.user_id;
END IF;
RETURN NEW;
END;
```

```
LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_set_audit_log_institution
BEFORE INSERT ON audit_logs
FOR EACH ROW EXECUTE FUNCTION set_audit_log_institution();
```

```
-- =====
=====
-- 23. CHAT HISTORY
-- =====
=====
CREATE TABLE chat_history (
    chat_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    student_id   UUID NOT NULL REFERENCES students (student_i
```

```

d) ON DELETE CASCADE,
    question    TEXT NOT NULL,
    response     TEXT,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_chat_history_student_id ON chat_history (stu
dent_id);

COMMIT;

-- =====
=====
-- END OF SCHEMA
--
-- Deliberately NOT changed (would require new tables, out of
scope
-- for this pass):
--   - document_types is still global, not per-institution/pe
r-course.
--   - students.category / seat_matrix.category / fee_structu
re.category
--       remain free-text, not a shared lookup table.
--   - No separate permissions table (RBAC is role-level onl
y).
--   - Guardian/parent has no login account, only contact fie
lds.
-- These are still worth doing eventually, but they need a ne
w table
-- each, which you asked to skip for now.
-- =====
=====

```